

# CSS brightness() and contrast()

A mathematical and implementation-oriented guide — from pixel values to affine color matrices, GPU shaders, chaining, clamping, and alpha

Purpose. This guide consolidates the supplied explanation, corrects terminology where needed, and fills the conceptual gaps between the CSS function, the per-channel equation, the 5×4 color matrix, and how a browser can execute the operation efficiently.

## 1. The mental model: a CSS filter is a pixel transformation

An image is ultimately processed as pixels. For a normal RGB pixel, think of the color as three normalized channel values: R, G and B, each in the interval [0, 1]. For example, an 8-bit red value of 128 corresponds approximately to  $128/255 = 0.502$ .

A CSS filter does not 'edit the picture' as a semantic object. Conceptually, it takes every input pixel and computes a new pixel. For brightness and contrast, the same mathematical rule is applied independently to R, G and B.

Important distinction: CSS filter mathematics is normally described using unpremultiplied color components. Actual rendering pipelines may use premultiplied-alpha representations internally, so the conceptual formula and the GPU storage representation should not be confused.

## 2. Why normalize 8-bit pixels to 0...1?

The CSS/filter equations are easiest to understand in normalized coordinates. The mapping is:

$$\text{normalized} = \text{8-bit} / 255 \quad \text{and} \quad \text{8-bit} \approx \text{normalized} \times 255$$

Thus black is 0, white is 1, and middle gray is 0.5 (about 128 in 8-bit terms). After the mathematical transformation, the result is clamped to the valid range before it is represented for display.

## 3. Brightness(B): the simplest case

The conceptual rule for filter: brightness(B) is a multiplier applied to every color channel:

$$C' = B \times C$$

Here C is one input channel (R, G, or B), B is the brightness factor, and C' is the output channel. In CSS, 100% means  $B = 1.0$ ; 50% means  $B = 0.5$ ; 0% means  $B = 0.0$ . Values above 100% use  $B > 1$ .

Example: input  $R = 0.40$  and brightness(150%) gives  $R' = 1.5 \times 0.40 = 0.60$ . The same calculation is performed for G and B.

**Because the result must remain a valid color component, conceptually:  $C' = \text{clamp}(B \times C, 0, 1)$ .**

## 4. Brightness as a color matrix

Brightness is a special case where no constant offset is required. In homogeneous/affine color-matrix notation, the RGB part can be represented as a 5×4 matrix acting on [R,G,B,A,1]. One common row layout is:

$$\begin{bmatrix} B & 0 & 0 & 0 & 0 \\ 0 & B & 0 & 0 & 0 \\ 0 & 0 & B & 0 & 0 \end{bmatrix}$$

[ 0 0 0 1 0 ]

The last column is the constant term. For brightness it is zero, because the operation is pure scaling. Alpha is shown unchanged in this conceptual matrix.

## 5. Contrast(K): scaling around a midpoint

Contrast is not simply  $C' = K \times C$ . That would make black stay black and would not describe the usual meaning of contrast. Instead, contrast scales the distance of a channel from a fixed midpoint.

$$C' = K(C - 0.5) + 0.5$$

Expanding it gives:

$$C' = K \times C + 0.5(1 - K)$$

This is the key bridge between the intuitive idea ('push values away from gray') and the matrix implementation ('multiply, then add a constant').

## 6. Understand contrast with three values

| K   | Equation           | Meaning                                          |
|-----|--------------------|--------------------------------------------------|
| 0   | $C' = 0.5$         | Every channel becomes middle gray.               |
| 0.5 | $C' = 0.5C + 0.25$ | Values move halfway toward 0.5.                  |
| 1   | $C' = C$           | No change.                                       |
| 2   | $C' = 2C - 0.5$    | Distances from 0.5 are doubled; values may clip. |

Notice what  $K = 0$  does: the slope becomes zero and the intercept becomes 0.5. Therefore all input values collapse to exactly 0.5.

## 7. Why contrast needs the 5th column

This is the most important linear-algebra gap to understand. A conventional 3×3 matrix can mix and scale R, G and B, but every output is still a combination of the input components. If R=G=B=0, a purely linear transformation always produces zero.

**Contrast, however, contains a constant term:  $0.5(1-K)$ . A constant does not come from R, G or B. Therefore we augment the input vector with a constant 1:**

$$\mathbf{P} = [\mathbf{R}, \mathbf{G}, \mathbf{B}, \mathbf{A}, 1]^T$$

The final matrix column supplies an input that is always exactly 1, allowing the matrix to add a constant offset to each channel.

This is called an affine transformation. The same mathematical idea is used in graphics for translation: homogeneous coordinates add an extra coordinate so that translation can be expressed using matrix multiplication.

## 8. Contrast matrix

**Let K be the contrast factor and  $O = 0.5(1-K)$ . The conceptual color matrix is:**

$$\begin{bmatrix} K & 0 & 0 & 0 & 0 \\ 0 & K & 0 & 0 & 0 \\ 0 & 0 & K & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

The red output, for example, is  $R' = K \cdot R + O$ . Substituting O gives exactly  $K \cdot R + 0.5(1-K)$ , which is the expanded contrast equation.

## 9. A geometric view: slope and intercept

Any per-channel operation of the form  $C' = mC + b$  can be understood as a straight line. Brightness has  $m=B$  and  $b=0$ . Contrast has  $m=K$  and  $b=0.5(1-K)$ .

| Operation     | Slope m | Intercept b | Geometric effect                |
|---------------|---------|-------------|---------------------------------|
| Brightness(B) | B       | 0           | Scale around black (0).         |
| Contrast(K)   | K       | $0.5(1-K)$  | Scale around middle gray (0.5). |

This geometric model explains almost everything: changing the slope changes the dynamic range, while changing the intercept repositions the line.

## 10. Clamping: why values cannot escape the color range

**Mathematics can produce values below 0 or above 1. A displayable normalized channel cannot. Therefore the transformed component is constrained:**

$$C_{\text{display}} = \min(1, \max(0, C'))$$

For example, `contrast(2)` applied to  $C=0$  gives  $C'=-0.5$ , which clamps to 0. Applied to  $C=1$ , it gives 1.5, which clamps to 1. This clipping is one reason aggressive contrast can destroy shadow/highlight detail.

## 11. Chaining filters: composition, not independent operations

If multiple filters are specified, each filter receives the output of the preceding filter. For example:

**filter: contrast(1.2) brightness(0.9);**

Start with  $C$ . First contrast:  $C_1 = 1.2C + 0.5(1-1.2) = 1.2C - 0.1$ . Then brightness:  $C_2 = 0.9C_1 = 1.08C - 0.09$ .

If the order is reversed, brightness first gives  $C_1 = 0.9C$  and then contrast gives  $C_2 = 1.2(0.9C) - 0.1 = 1.08C - 0.1$ . The slope is the same here, but the intercept differs. Therefore the output can differ. In general, affine transformations do not commute.

## 12. Combining matrices

Each filter can be represented as a matrix. A chain can therefore be represented by matrix multiplication. The exact multiplication convention depends on whether vectors are treated as column or row vectors, but the core idea is the same: the later transformation consumes the earlier transformation's output.

This is useful for performance because several compatible color operations can potentially be combined into one color-matrix operation instead of applying separate passes.

## 13. What the GPU/shader sees

A browser does not have to literally execute a JavaScript loop over every pixel. A GPU fragment shader can perform the same per-pixel arithmetic in parallel. Conceptually, a shader receives a sampled pixel, performs the matrix/vector operation, clamps the result as required by the rendering pipeline, and writes the resulting color.

**A simplified GLSL-style conceptual operation is:**

```
vec4 p = texture(image, uv);
float r = p.r * K + O;
float g = p.g * K + O;
float b = p.b * K + O;
```

For a general  $5 \times 4$  color matrix, the shader can instead calculate each output component as a dot product with the extended vector  $[r, g, b, a, 1]$ . Real browser implementations are more complex and may use implementation-specific optimizations, color-management stages, intermediate surfaces, and GPU APIs; the shader above is a conceptual model, not a claim about a particular browser's source code.

## 14. Alpha: the subtle part

RGB filter equations should not be confused with alpha handling. In an unpremultiplied conceptual model, R/G/B are independent color components and alpha is separate. In a premultiplied representation, RGB values have already been multiplied by alpha. Rendering engines commonly use premultiplied alpha internally because it makes compositing behave correctly around transparency.

Therefore, if you implement a filter yourself in Canvas/WebGL, you must know whether your texture/data is premultiplied and whether the API converts it for you. Applying an unpremultiplied formula directly to premultiplied values can produce incorrect edge colors for transparent pixels.

## 15. A concrete pixel walkthrough

Suppose a pixel is RGB = (64, 128, 192) in 8-bit form. Normalize it: approximately (0.251, 0.502, 0.753).

Brightness 120%: multiply each component by 1.2 → approximately (0.301, 0.602, 0.904), then convert back to 8-bit if needed → approximately (77, 153, 231).

Contrast 120%: use  $C' = 1.2C - 0.1$  → approximately (0.201, 0.502, 0.804) → approximately (51, 128, 205).

The contrast result illustrates the midpoint rule: a channel near 0.5 barely moves, while channels farther below or above 0.5 move farther away.

## 16. The key differences in one table

| Property      | brightness(B)    | contrast(K)              |
|---------------|------------------|--------------------------|
| Core equation | $C' = B \cdot C$ | $C' = K(C - 0.5) + 0.5$  |
| Expanded      | $B \cdot C$      | $K \cdot C + 0.5(1 - K)$ |
| Pivot         | 0 (black)        | 0.5 (middle gray)        |
| Slope         | B                | K                        |
| Intercept     | 0                | $0.5(1 - K)$             |
| Needs offset? | No               | Yes                      |

|                        |                     |                            |
|------------------------|---------------------|----------------------------|
| Matrix constant column | 0                   | $0.5(1-K)$                 |
| At 0                   | Black               | Middle gray                |
| At 1                   | Original            | Original                   |
| Above 1                | Brighter; may clamp | Higher contrast; may clamp |

## 17. Important corrections and terminology

- A  $5 \times 4$  color matrix is a common way to describe CSS-style color transformations using four input components plus a constant term. It is more precise to think of it as a 4-output-by-5-input affine transform than simply 'a  $5 \times 4$  matrix because GPUs need it'.
- The extra constant coordinate is a mathematical homogeneous-coordinate device; GPUs do not require it specifically for CSS. Graphics hardware supports vector/matrix arithmetic, while the affine representation is a convenient mathematical encoding.
- `brightness()` itself does not require a translation term. `contrast()` does because its pivot is 0.5.
- `hue-rotate()` is fundamentally a channel-mixing linear transformation; it does not inherently require a constant offset in the same way `contrast()` does. Avoid generalizing that every CSS filter needs the fifth column for a constant.
- Actual browser rendering is governed by the CSS Filter Effects specification and the browser's color-management/rendering pipeline. The equations here are the useful conceptual model for implementation and reasoning.

## 18. How to reason about any similar filter

When you encounter a new image transformation, use this sequence:

|   |                         |                                                                       |
|---|-------------------------|-----------------------------------------------------------------------|
| 1 | Define the input domain | Are values 0...1, 0...255, linear-light, or another color space?      |
| 2 | Write one-channel math  | Start with $C$ and derive $C'$ .                                      |
| 3 | Put it in $mC+b$ form   | This exposes slope and constant offset.                               |
| 4 | Check the pivot         | Is the operation centered on black, gray, white, or another point?    |
| 5 | Convert to matrix form  | Use channel-mixing coefficients plus the constant column when needed. |
| 6 | Check clamping          | Identify where information is clipped.                                |
| 7 | Check alpha             | Determine premultiplied/unpremultiplied behavior.                     |
| 8 | Check chaining          | Compose operations in the specified order.                            |

## 19. Final mental picture

Brightness: take the color value and stretch/shrink it relative to black.

Contrast: take the color value, measure its distance from middle gray, scale that distance, then put it back around middle gray.

Matrix: express those arithmetic operations as a vector transformation.

5th column: provide the constant '1' that lets the matrix add an offset.

GPU: apply the same small amount of arithmetic independently to huge numbers of pixels in parallel.

Clamping: force results back into the valid display range.

Chaining: feed one transformation's output into the next, preserving order.

## Reference note

For standards-level implementation, consult the current W3C Filter Effects specification and browser documentation rather than relying on simplified educational formulas alone. The purpose of this

document is to make the mathematical structure understandable and implementation-ready, not to reproduce browser source code.